



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

**Constituent of Symbiosis International (Deemed University),
Pune**

Subject: Computer Vision Laboratory

Experiment - 8

Aim: To detect human faces using Haar cascades or deep learning models.

Software Tools Required: VS Code, Python 3.12

Theory:

1. Face Detection Using Haar Cascades

Overview

Haar Cascade is a machine learning-based approach for object detection, introduced by Paul Viola and Michael Jones in 2001. It is one of the classical methods used for real-time face detection.

How It Works

1. Haar-like Features

- Haar features are simple rectangular features that encode the difference in intensity between adjacent rectangular regions.
- These features capture characteristics like edges, lines, and textures, which are useful for distinguishing a face from the background.
- **Examples:**
 - Edge feature: Difference between the sum of pixels in two adjacent rectangles.
 - Line feature: Difference among three rectangles.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

2. Integral Image

- To compute Haar features quickly, an integral image is used.
- The integral image allows fast calculation of the sum of pixel values in any rectangular area of the image.

Constituent of Symbiosis International (Deemed University), Pune

3. Training with AdaBoost

- A large set of positive (faces) and negative (non-faces) images is used for training.
- AdaBoost selects a small number of critical features and combines them into a strong classifier.

4. Cascade of Classifiers

- Instead of applying a single classifier to the whole image, multiple classifiers are arranged in a cascade.
- Early stages quickly discard non-face regions. ○ Later stages perform more detailed checks to confirm the presence of a face.
- This improves detection speed and reduces false positives.

Advantages

- Fast and efficient, suitable for real-time detection.
- Simple to implement using OpenCV (haarcascade_frontalface_default.xml).

Limitations

- Sensitive to lighting conditions, face orientation, and occlusion.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

- Not as accurate as modern deep learning methods.

2. Face Detection Using Deep Learning Models

Overview

Deep learning models for face detection leverage Convolutional Neural Networks (CNNs) to automatically learn hierarchical features from images. These methods are more robust than classical Haar cascades.

Popular Models

1. SSD (Single Shot MultiBox Detector)

Constituent of Symbiosis International (Deemed University), Pune

- Detects faces at multiple scales.
- Efficient for real-time detection.

2. MTCNN (Multi-task CNN)

- Detects faces and facial landmarks (eyes, nose, mouth).
- Consists of three networks: P-Net, R-Net, and O-Net.

3. Faster R-CNN

- Region-based CNN that generates proposals and classifies them.
- More accurate but slower than SSD/MTCNN.

4. YOLO (You Only Look Once)

- Single network predicts bounding boxes and class probabilities.
- Very fast, suitable for video processing.

How It Works



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

1. Feature Extraction

- CNN layers automatically extract features like edges, textures, and shapes.

2. Bounding Box Prediction

- The model predicts the coordinates of face regions in the image.

3. Classification

- Each predicted box is classified as face or background.

4. Post-processing

- Techniques like Non-Maximum Suppression (NMS) remove overlapping detections and reduce false positives.

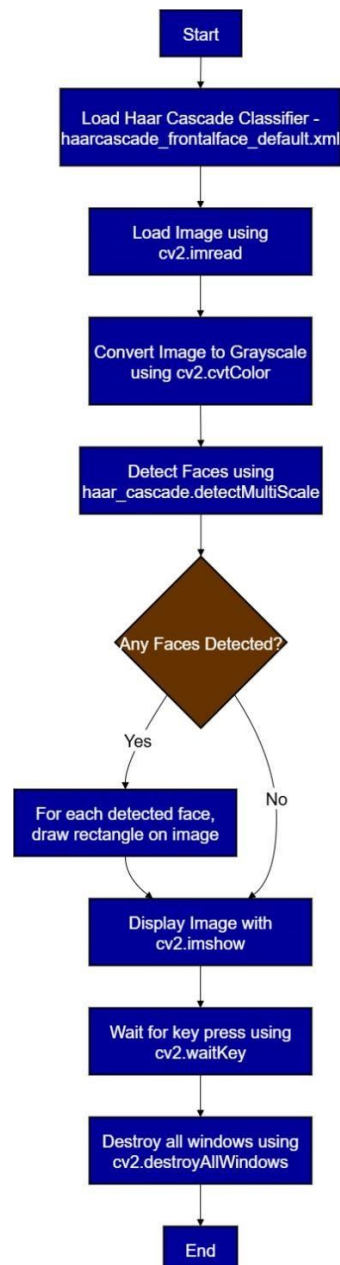
**Constituent of Symbiosis International (Deemed University),
Pune**

Flowchart Part-

1:



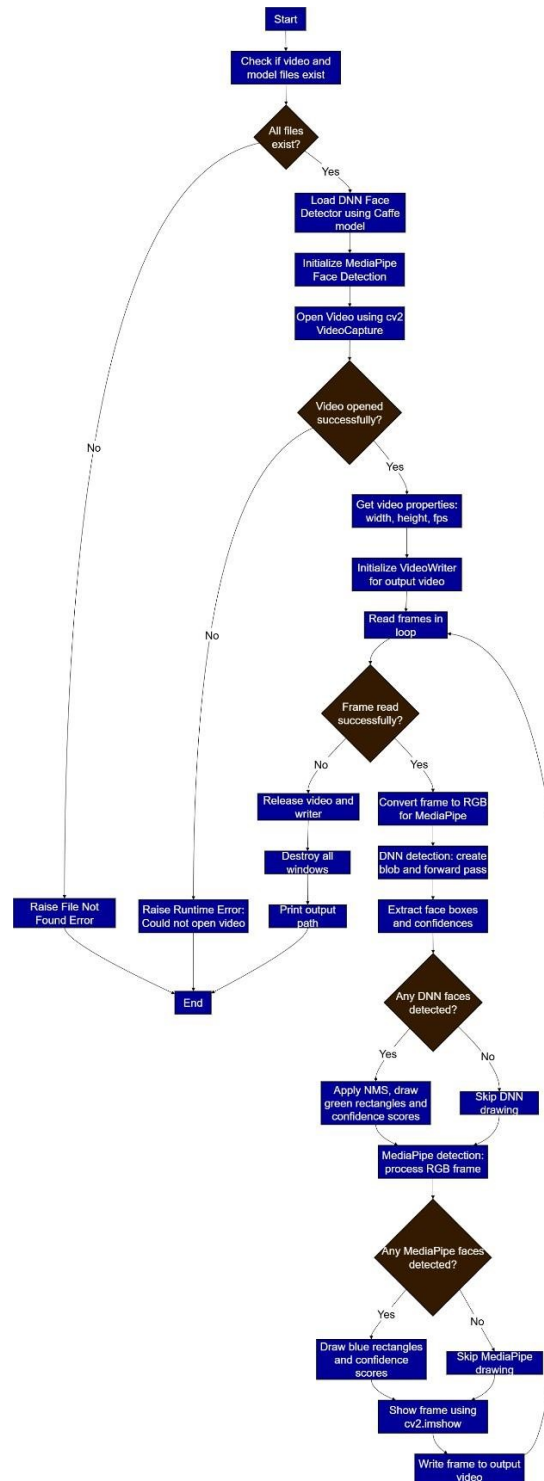
SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE



**Constituent of Symbiosis International (Deemed University),
Pune**
Part-2:



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE





SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune Program with Output

Program:

Part-1

```
import cv2

# Load Haar cascade classifier
haar_cascade = cv2.CascadeClassifier(cv2.data.harcascades + "haarcascade_frontalface_default.xml")

# Load image
image = cv2.imread("Images.png")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Detect faces
faces = haar_cascade.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=5,
    minSize=(30, 30)
)

# Draw bounding boxes
for (x, y, w, h) in faces:
    cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 2)

# Display the result
cv2.imshow("Haar Cascade Face Detection", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Part-2

```
import cv2
import numpy as np
import os
import mediapipe as mp

# --- Paths ---
video_path = r"E:/Computer Vision/Walk_2.mp4"
out_file = r"E:/Computer Vision/human_faces_all.mp4"
modelFile = r"E:/Computer Vision/res10_300x300_ssd_iter_140000.caffemodel"
configFile = r"E:/Computer Vision/deploy.prototxt"

# --- Check files exist ---
for path in [video_path, modelFile, configFile]:
    if not os.path.exists(path):
        raise FileNotFoundError(f"File not found: {path}")

# --- Load DNN face detector ---
net = cv2.dnn.readNetFromCaffe(configFile, modelFile)

# --- MediaPipe Face Detection ---
mp_face = mp.solutions.face_detection
```

Name-Janay Rawal
PRN-22070123049
ENTC-A G1



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

**Constituent of Symbiosis International (Deemed University),
Pune**

Department of Electronics and Telecommunication Engineering



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

```
# --- Open video ---
cap = cv2.VideoCapture(video_path)
if not cap.isOpened():
    raise RuntimeError("Could not open video.")

# --- Video properties ---
w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# --- Video writer ---
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
out = cv2.VideoWriter(out_file, fourcc, fps, (w, h))

while True:
    ret, frame = cap.read()
    if not ret:
        break

    h_frame, w_frame = frame.shape[:2]
    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    # --- DNN detection ---
    blob = cv2.dnn.blobFromImage(frame, 1.0, (300, 300),
                                   [104, 117, 123], swapRB=False, crop=False)
    net.setInput(blob)
    detections = net.forward()

    face_boxes = []
```

```
face_boxes = []
confidences = []

for i in range(detections.shape[2]):
    confidence = detections[0, 0, i, 2]
    if confidence > 0.3:
        box = detections[0, 0, i, 3:7] * np.array([w_frame, h_frame, w_frame, h_frame])
        x1, y1, x2, y2 = box.astype("int")
        face_boxes.append([x1, y1, x2 - x1, y2 - y1])
        confidences.append(float(confidence))

# --- Apply NMS for DNN ---
if len(face_boxes) > 0:
    indices = cv2.dnn.NMSBoxes(face_boxes, confidences, 0.3, 0.4)
    for i in indices.flatten():
        x, y, w_box, h_box = face_boxes[i]
        cv2.rectangle(frame, (x, y), (x + w_box, y + h_box), (0, 255, 0), 2)
        cv2.putText(frame, f"{confidences[i]:.2f}", (x, y - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1)

# --- MediaPipe detection ---
results_mp = face_detection_mp.process(frame_rgb)
if results_mp.detections:
    for detection in results_mp.detections:
        bboxC = detection.location_data.relative_bounding_box
```

**Constituent of Symbiosis International (Deemed University),
Pune**



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

```
# --- MediaPipe detection ---
results_mp = face_detection_mp.process(frame_rgb)
if results_mp.detections:
    for detection in results_mp.detections:
        bboxC = detection.location_data.relative_bounding_box
        x1 = int(bboxC.xmin * w_frame)
        y1 = int(bboxC.ymin * h_frame)
        w_box = int(bboxC.width * w_frame)
        h_box = int(bboxC.height * h_frame)
        cv2.rectangle(frame, (x1, y1), (x1 + w_box, y1 + h_box), (255, 0, 0), 2)
        score = int(detection.score[0] * 100)
        cv2.putText(frame, f"{score}%", (x1, y1 - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 1)

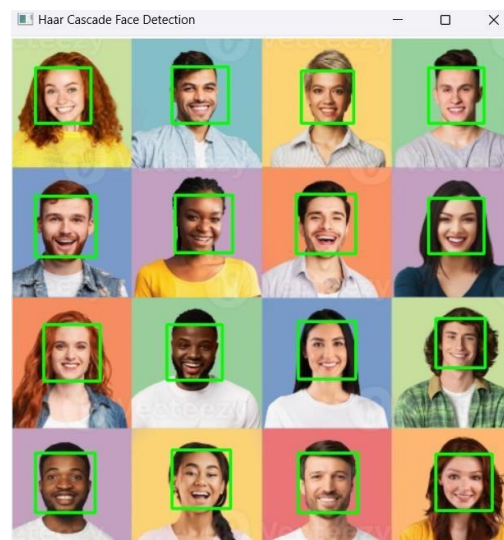
# --- Show and save frame ---
cv2.imshow("All Human Faces Detection", frame)
out.write(frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
out.release()
cv2.destroyAllWindows()
print(f"Output video saved at: {out_file}")
```

Output:

Part-1



**Constituent of Symbiosis International (Deemed University),
Pune**



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Part-2



Conclusion/Inferences:

Human face detection is a fundamental task in computer vision with applications ranging from security and surveillance to human-computer interaction. Haar cascades provide a fast and lightweight solution suitable for real-time detection in controlled environments, while deep learning models, such as DNN-based detectors or MediaPipe, offer higher accuracy and robustness under varying lighting conditions, poses, and occlusions. Combining traditional methods with modern deep learning approaches can enhance detection reliability, making face detection more adaptable to diverse real-world scenarios. Overall, the choice of method depends on the specific requirements for speed, accuracy, and computational resources.